

Oracle SQL

Indexing

Complete Reference Guide

A comprehensive guide to every Oracle SQL index type — with syntax, practical examples, explain-plan tips, and maintenance commands.

Index Types Covered	Explain Plan	Bitmap & Full-Text	Maintenance
10	✓	✓	✓

Table of Contents

- 0. Setup — Base Tables
- 1. Single-Column Index
- 2. Unique Index
- 3. Composite Index
- 4. Primary Key Index (Clustered)
- 5. Full-Text Index (Oracle Text)
- 6. Partial Index (Function-Based Workaround)
- 7. Covering Index
- 8. Function-Based (Expression) Index
- 9. Bitmap Index
- 10. Index Monitoring & Maintenance
- 11. Quick Reference Table

0. Setup — Base Tables

Run the following DDL and DML once before executing any of the index examples below. All sections reference the employees and orders tables.

```
-- employees table

CREATE TABLE employees (

emp_id NUMBER PRIMARY KEY,

first_name VARCHAR2(50),

last_name VARCHAR2(50),

email VARCHAR2(100),

department VARCHAR2(50),

salary NUMBER,

status VARCHAR2(10),

hire_date DATE,

manager_id NUMBER

);


-- orders table

CREATE TABLE orders (

order_id NUMBER PRIMARY KEY,

customer_id NUMBER,

order_date DATE,

amount NUMBER,

region VARCHAR2(20)

);


-- Sample data

INSERT INTO employees VALUES (1,'Alice','Smith','alice@mail.com','HR',60000,'active',DATE
'2020-01-15',101);

INSERT INTO employees VALUES (2,'Bob','Jones','bob@mail.com','IT',80000,'active',DATE
'2019-03-22',102);

INSERT INTO employees VALUES (3,'Carol','Smith','carol@mail.com','HR',55000,'inactive',DATE
'2021-07-10',101);

INSERT INTO employees VALUES (4,'Dave','Brown','dave@mail.com','IT',90000,'active',DATE
'2018-11-05',102);
```

```
INSERT INTO employees VALUES (5, 'Eve', 'White', 'eve@mail.com', 'Finance', 70000, 'active', DATE
'2022-02-28', 103);

INSERT INTO orders VALUES (1001, 1, DATE '2024-01-10', 500, 'North');

INSERT INTO orders VALUES (1002, 2, DATE '2024-02-15', 1500, 'South');

INSERT INTO orders VALUES (1003, 1, DATE '2024-03-20', 750, 'North');

INSERT INTO orders VALUES (1004, 3, DATE '2024-04-05', 200, 'East');

INSERT INTO orders VALUES (1005, 2, DATE '2024-05-18', 3000, 'West');

COMMIT;
```

1. Single-Column Index

The most common index type. Speeds up queries that filter on a single column by building a B-Tree structure over that column's values.

■ Look for 'INDEX RANGE SCAN' in the DBMS_XPLAN output to confirm usage.

```
-- Create index on last_name

CREATE INDEX idx_emp_lastname ON employees(last_name);

-- Query that uses the index

SELECT emp_id, first_name, last_name, department
FROM employees
WHERE last_name = 'Smith';

-- Verify via Explain Plan (look for INDEX RANGE SCAN)

EXPLAIN PLAN FOR

SELECT emp_id, first_name FROM employees WHERE last_name = 'Smith';

SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);
```

2. Unique Index

Ensures no duplicate values exist in the indexed column. Oracle automatically creates a unique index for every PRIMARY KEY and UNIQUE constraint.

```
-- Enforce no duplicate emails

CREATE UNIQUE INDEX idx_unique_email ON employees(email);

-- This INSERT succeeds

INSERT INTO employees

VALUES (6, 'Frank', 'Green', 'frank@mail.com', 'IT', 72000, 'active', DATE '2023-01-01', 102);

-- This INSERT raises ORA-00001: unique constraint violated

INSERT INTO employees

VALUES (7, 'Ghost', 'User', 'alice@mail.com', 'HR', 50000, 'active', DATE '2023-06-01', 101);

-- Single-row fetch (INDEX UNIQUE SCAN)

SELECT * FROM employees WHERE email = 'bob@mail.com';
```

3. Composite Index (Multi-Column)

An index on two or more columns. Column order is critical — Oracle can use the index only when the query references the leftmost column(s).

■ Always filter by the leftmost column first. This is called the 'leading column' rule.

```
CREATE INDEX idx_lastname_dept ON employees(last_name, department);

-- Uses index: leftmost column present

SELECT * FROM employees WHERE last_name = 'Smith';

-- Uses index: both columns present

SELECT * FROM employees WHERE last_name = 'Smith' AND department = 'HR';

-- Does NOT use index: skips leftmost column

SELECT * FROM employees WHERE department = 'HR';

-- Verify

EXPLAIN PLAN FOR
```

```
SELECT * FROM employees WHERE last_name = 'Smith' AND department = 'HR';

SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);
```

4. Primary Key Index (Clustered)

Oracle automatically creates a unique B-Tree index when you define a PRIMARY KEY. This is the fastest possible lookup — Oracle performs an INDEX UNIQUE SCAN and retrieves the row in a single I/O.

```
-- Auto-created by Oracle on table creation

-- emp_id and order_id already have primary key indexes

-- Inspect auto-created indexes
SELECT index_name, index_type, uniqueness
FROM user_indexes
WHERE table_name = 'EMPLOYEES';

-- Direct PK lookup – INDEX UNIQUE SCAN
SELECT * FROM employees WHERE emp_id = 3;

EXPLAIN PLAN FOR SELECT * FROM employees WHERE emp_id = 3;

SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);

-- Plan shows: INDEX UNIQUE SCAN on SYS_C...
```

5. Full-Text Index (Oracle Text)

Oracle Text provides full-text search capabilities over VARCHAR2, CLOB, and BLOB columns. It is far more powerful than LIKE '%word%', which always causes a full table scan.

■ Requires the CTXSYS schema to be installed (standard in Oracle DB Enterprise Edition).

```
CREATE TABLE articles (
  article_id NUMBER PRIMARY KEY,
  title VARCHAR2(200),
  body CLOB
);

INSERT INTO articles VALUES (1, 'Oracle Indexing Guide',
  'Covers all index types in Oracle including B-Tree and bitmap');
```

```

INSERT INTO articles VALUES (2,'Database Performance Tips',
'Tune your Oracle database for better performance and indexing.');
```

```

COMMIT;
```

```

-- Create Oracle Text (full-text) index

CREATE INDEX idx_fulltext_articles ON articles(body)

INDEXTYPE IS CTXSYS.CONTEXT;
```

```

-- Basic search

SELECT article_id, title

FROM articles

WHERE CONTAINS(body, 'indexing') > 0;
```

```

-- Boolean search

SELECT article_id, title

FROM articles

WHERE CONTAINS(body, 'Oracle AND performance') > 0;
```

```

-- With relevance score

SELECT article_id, title, SCORE(1) AS relevance

FROM articles

WHERE CONTAINS(body, 'Oracle', 1) > 0

ORDER BY SCORE(1) DESC;
```

6. Partial Index (Function-Based Workaround)

Oracle does not support a WHERE clause in CREATE INDEX directly. The standard workaround is a function-based index using a CASE expression that returns NULL for rows you want to exclude (Oracle never indexes NULL values).

■ *NULL values are never stored in a B-Tree index, so excluded rows consume no index space.*

```

-- Index only active employees

CREATE INDEX idx_active_email

ON employees(CASE WHEN status = 'active' THEN email END);
```

```

-- Query that matches the expression

SELECT emp_id, first_name, email
```

```
FROM employees

WHERE CASE WHEN status = 'active' THEN email END = 'alice@mail.com';
```

Alternative: Virtual Column + Index

```
-- Add virtual column

ALTER TABLE employees ADD (

active_flag AS (CASE WHEN status = 'active' THEN emp_id ELSE NULL END)

);

-- Index the virtual column

CREATE INDEX idx_active_empid ON employees(active_flag);

-- Fast query on active employees only

SELECT * FROM employees WHERE active_flag IS NOT NULL;
```

7. Covering Index

A covering index includes every column a query needs. Oracle can satisfy the entire query from the index structure alone — no visit to the table (heap) is needed. This eliminates a second I/O lookup and is a major performance technique.

■ If the plan shows 'TABLE ACCESS BY INDEX ROWID', the index is not fully covering.

```
-- Include all columns the query will SELECT or filter on

CREATE INDEX idx_covering_orders

ON orders(customer_id, order_date, amount);

-- Query is 100% served from the index

SELECT order_date, amount

FROM orders

WHERE customer_id = 2;

EXPLAIN PLAN FOR

SELECT order_date, amount FROM orders WHERE customer_id = 2;

SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);

-- Look for INDEX RANGE SCAN with NO TABLE ACCESS row below it
```

8. Function-Based (Expression) Index

Indexes the result of a SQL function or arithmetic expression on a column. Allows Oracle to use an index even when the WHERE clause applies a transformation. The query must use the exact same expression as the index definition.

■ Without the function-based index, `WHERE UPPER(email) = '...'` forces a full table scan.

```
-- Case-insensitive email lookup

CREATE INDEX idx_upper_email ON employees(UPPER(email));

-- Uses the index

SELECT * FROM employees WHERE UPPER(email) = 'ALICE@MAIL.COM';

-- Also works with mixed-case input

SELECT * FROM employees WHERE UPPER(email) = UPPER('Alice@Mail.Com');

-- Another example: TRUNC on salary

CREATE INDEX idx_salary_range ON employees(TRUNC(salary, -4));

SELECT * FROM employees WHERE TRUNC(salary, -4) = 60000;
```

9. Bitmap Index

A bitmap index stores a bit string for each distinct value of the indexed column. It excels on low-cardinality columns (status, department, region) in read-heavy data warehouse environments. Oracle can combine multiple bitmap indexes using fast bitwise AND/OR operations.

■ **Never use Bitmap indexes on OLTP tables with frequent INSERT/UPDATE/DELETE. They lock the entire bitmap segment and cause severe row-level lock contention.**

```
-- Best for low-cardinality, read-heavy tables

CREATE BITMAP INDEX idx_bitmap_status ON employees(status);

CREATE BITMAP INDEX idx_bitmap_dept ON employees(department);

-- Oracle combines bitmaps with AND internally

SELECT COUNT(*)

FROM employees

WHERE status = 'active'

AND department = 'IT';
```

```
-- Multi-filter query

SELECT emp_id, first_name

FROM employees

WHERE status = 'active'

AND department = 'HR';


EXPLAIN PLAN FOR

SELECT COUNT(*) FROM employees WHERE status='active' AND department='IT';

SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);

-- Look for: BITMAP AND operation in the plan
```

10. Index Monitoring & Maintenance

Over time, indexes can become fragmented due to row churn. Oracle provides DDL commands to inspect, rebuild, and manage indexes without data loss.

```
-- List all indexes on a table

SELECT index_name, index_type, uniqueness, status

FROM user_indexes

WHERE table_name = 'EMPLOYEES';

-- Show columns in each index

SELECT index_name, column_name, column_position

FROM user_ind_columns

WHERE table_name = 'EMPLOYEES'

ORDER BY index_name, column_position;

-- Rebuild a fragmented index (table lock)

ALTER INDEX idx_emp_lastname REBUILD;

-- Rebuild online – safe for production (no table lock)

ALTER INDEX idx_emp_lastname REBUILD ONLINE;

-- Mark unusable (equivalent to disabling)

ALTER INDEX idx_emp_lastname UNUSABLE;

-- Re-enable by rebuilding

ALTER INDEX idx_emp_lastname REBUILD;

-- Drop an index permanently

DROP INDEX idx_emp_lastname;
```

11. Quick Reference Table

Index Type	Oracle Syntax	Range?	Best Use Case	Notes
Single-Column	CREATE INDEX	Yes	General WHERE filters	Most common
Unique	CREATE UNIQUE INDEX	Yes	Email, username, SSN	Auto on PK/UNIQUE

Index Type	Oracle Syntax	Range?	Best Use Case	Notes
Composite	<code>CREATE INDEX ON (c1,c2)</code>	Yes	Multi-column WHERE/JOIN	Left-prefix rule
Primary Key	<code>AUTO on PRIMARY KEY</code>	Yes	PK lookups	1 per table
Full-Text	<code>INDEXTYPE IS CTXSYS.CONTEXT</code>	No	CLOB / text search	CONTAINS() syntax
Partial	<code>CASE WHEN expression</code>	Yes	Active/filtered rows	NULL not indexed
Covering	<code>All cols in index</code>	Yes	Read-heavy, no heap hit	No TABLE ACCESS step
Function-Based	<code>CREATE INDEX ON (FUNC(col))</code>	Yes	UPPER, TRUNC, expressions	Exact expr match
Bitmap	<code>CREATE BITMAP INDEX</code>	No	DW, low-cardinality cols	OLTP: avoid!

When NOT to Index

- Small tables — a full scan is faster than index overhead.
- Low-cardinality columns with very few distinct values (e.g., a boolean flag) — unless using Bitmap.
- Columns written to very frequently — every INSERT/UPDATE/DELETE must also update the index.
- Columns rarely used in WHERE, JOIN, or ORDER BY clauses.